

字节 AI 全栈挑战赛

ByteDance AI Full-Stack Challenge

AgentHub

多 Agent 协作平台设计

AI 协作开发记录与规范

AI Collaboration Record & Conventions

队名 启灵

队员 齐紫瀚 熊子恒 张桐铖

日期 2026 年 6 月

对应考察维度「AI 协作能力 (30%): 沉淀出和 AI 协作的 Spec、Skill、Rules 等协作规范」。

本项目 AgentHub 部分全程采用 Claude Code 结对开发。本文不是方法论汇编, 而是真实发生过的协作过程: 每条规范都给出它解决过的具体问题, 每个案例都可在 git 历史与代码注释中指认, 「真实协作会话档案」一节的统计直接来自本机 `~/ .claude/projects` 的会话日志。状态标注沿用全文统一图例: 已实现 已落地 · 部分实现 部分实现 · 路线图 路线图。

Contents

1	协作模型总述	4
2	Rules: 固化下来的硬约束及其实际作用	5
2.1	全局规则 (~/.claude/CLAUDE.md, 每次会话强制加载)	6
2.2	回合内追加的局部约束 (用户在对话中下达, AI 必须遵守)	6
3	Spec / Plan: 规格先行的固定格式	6
4	Skill: 工具链与各自的真实战绩	7
4.1	内置协作机制的使用 (Claude Code / memoh)	7
5	三个深度协作案例	8
5.1	案例一 (R7): 同一个 bot 单聊能跑、编排零产出——三层根因的逐层剥离	8
5.2	案例二 (R9): AI 评审纠正 AI 的方案	9
5.3	案例三: 一次「像 bug 的非 bug」——证据链排除法	9
6	失败与教训 (同样是沉淀)	9
7	量化记录 (可在 <code>git log</code> 验证)	10
8	真实协作会话档案	10
8.1	协作时间线	10
8.2	会话强度 (工具调用数, 按时间)	11
8.3	代表性人类指令 (原文摘录)	11
8.4	协作模式观察	11
9	产品设计概述 (精简)	12
9.1	背景与群聊心智	12
9.2	多 Agent 失效模式与应对	12
9.3	目标用户与典型场景	12
9.4	核心体验决策	12
10	系统架构与技术实现 (精简)	13
10.1	分层架构	13
10.2	数据模型与事件溯源	13
10.3	状态机与调度	14
10.4	双层 Planner 与适配器	14
10.5	设计 vs 实现对照	15
10.6	已知局限	15
11	沉淀给后来者的协作清单	15
12	结语	15

参考文献

16

1 协作模型总述

人工智能在软件开发中的使用不是简单的自动化, 而是人与模型之间的结对开发。开发者承担「导航员」角色, 负责把握方向、制定规则并验证结果;AI 扮演「驾驶员」, 在约束内快速完成调研、设计与编码。关于人-AI 协同的实践观察指出: 与生成式模型配对时, 开发者应专注于规划、架构与审查, 而把具体实现交给模型去草拟——「AI 的工作是快, 你的工作是好」¹。为保证 AI 在框架内工作,AgentHub 通过 Rules 与流程把这种分工制度化。

人 (开发者) 负责	AI(Claude Code) 负责
定边界与硬约束 (Rules)	代码调研与根因定位 (读代码、查日志、dump 事件)
每回合定目标与验收口径	出 Spec/Plan(根因 -> 设计 -> 验证 -> 风险)
对 Plan 行使否决 / 修订权	最小化实现 + 自测
提供环境事实 (部署、凭证)	浏览器实测验收 (Playwright)
决定「做 / 不做 / 回滚」	提交说明: 把 why 写进 commit message

Table 1: 人类与 AI 的分工。双向纠偏: 人纠 AI(见 §6 教训),AI 的二次评审也纠人选的方案 (见 §5.2 案例二)。

核心机制是回合制: 每个回合 (R1-R9) 一个明确目标, 流程为「调研 -> Plan -> 实现 -> 实测 -> 一个独立 commit」。回合间用户可随时插入新约束或改变优先级,AI 按最新约束执行。

¹观点见 Stack Overflow 博客关于 AI 结对编程的讨论: 人应担任导航员, 负责规划与审核,AI 负责生成代码。参见文末参考文献 [1]。

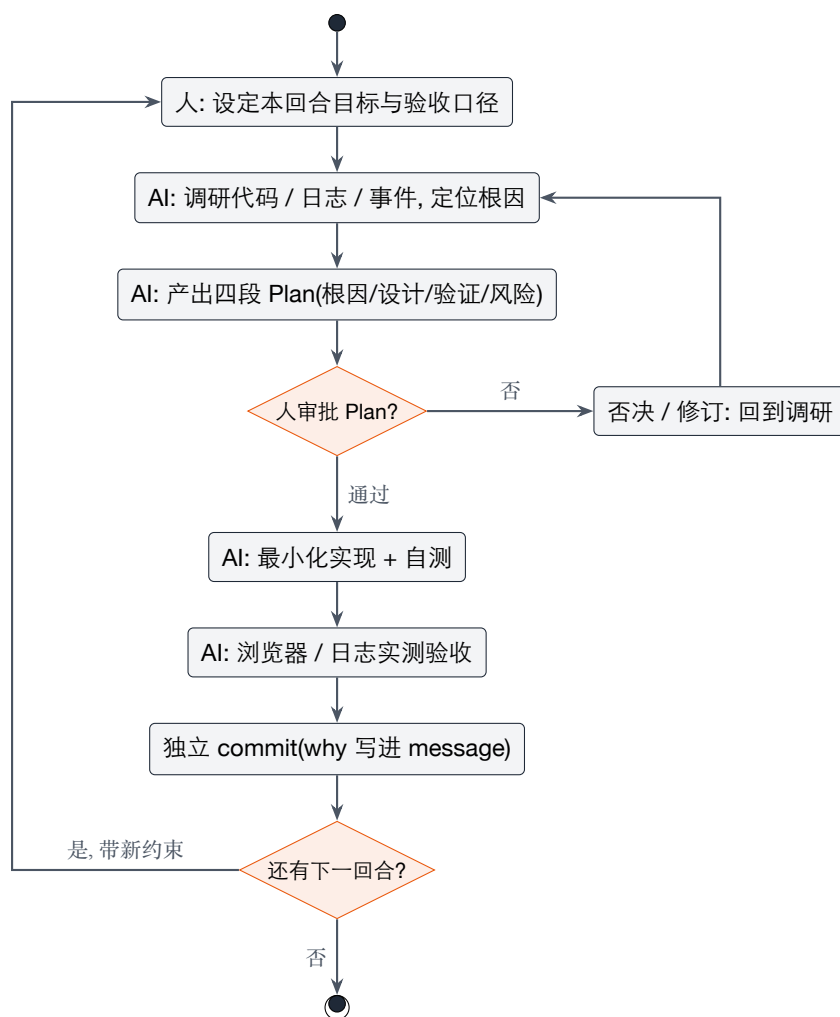


Figure 1: 图 1:AI-人回合式协作活动图。Plan 未过则回到调研; 每回合以独立 commit 收尾。

2 Rules: 固化下来的硬约束及其实际作用

开发者为 AI 设置明确的限制与验收条件, 本质是 **guardrail**: 用精确的要求与全面的测试约束模型, 避免其幻觉或越权调用²。

²guardrail(护栏) 指在人机配对开发中, 开发者通过明确边界与验收测试约束模型行为。参见文末参考文献 [3]。

2.1 全局规则 (~/.claude/CLAUDE.md, 每次会话强制加载)

规则原文	它实际防住了什么
不要轻易简化程序; 如需简化必须提前商量	R9 设计时 AI 曾倾向「只刷快轮询频率」的省事方案, 被此规则约束改为先 dump 真实事件流验证可行性再设计
如无必要, 勿增实体	R9 最终方案零新增后端端点 / 表 / 通道, 复用已有 GET /runs/:id/events;workspace 误报事件 (§5.3) 最终结论是「不改代码, 只加一行 gitignore」
不要偷懒	每回合强制以浏览器实测收尾, 不允许「代码看起来对了」就交差
Python 必须用虚拟环境	保证实测脚本环境一致性

2.2 回合内追加的局部约束 (用户在对话中下达,AI 必须遵守)

- 「chat 和 agenthub 不一样, 不改 chat 只修 agenthub」 -> 催生了接口反转设计:agenthub 定义 MemohRunner 最小接口、组合根注入 flow.Resolver, 一行没动 internal/conversation/flow(memoh.go 注释里写明了这个约束);
- 「不用 codex; 不依赖 ANTHROPIC_API_KEY; 凭证只从通用设置的 DeepSeek 走, 配置正确就不应报错」 -> 直接定义了 R4 的验收标准, 包括「反证实验」(停用 Provider 必须得到指路型错误而非 panic);
- 「完成一个 commit 一次」 -> 这条是用真实损失换来的, 见 §6。

3 Spec / Plan: 规格先行的固定格式

复杂改动一律先出 Plan、经用户确认再动手。可靠修复的前提是根因分析——先定义问题, 再收集数据、分析因果、提出方案³。Plan 固定四段:

- (1) 根因

必须代码级指认, 带文件:行号
- (2) 设计

改动清单 + 复用的现成件 + 不改什么
- (3) 验证

正向步骤 + 反证实验(故意触发错误, 确认不 panic 而是友好报错)
- (4) 风险/备注

已知坑、运维注意

真实样例 (R4 plan,~/.claude/plans/plan-resilient-sutherland.md 摘录):

根因 (已代码追踪确认): 同一个 cc bot, 单聊能跑、编排报错, 因为两条路读凭证的来源不同。单聊在 botruntime/cli.go 经 resolveCreds 从 DB 取「通用设置」凭证……编排只调 FromEnvWithDefaults()(纯读环境变量) 且 claudecode.go:83 硬性要求 APIKey……

验证:……反证: 临时停用通用设置里的 anthropic provider -> cc 子任务返回指向通用设置的友好错误 (非 panic、非 ANTHROPIC_API_KEY 字样),run 走「降级, 继续执行其余任务」。

³根因分析 (Root Cause Analysis) 是一种系统性流程: 定义问题 -> 收集数据 -> 分析因果 -> 提出并验证解决方案。参见文末参考文献 [2]。

价值: 动手前人就能审「根因对不对、改的是不是最小面、怎么算修好」,AI 不会拿着错误诊断写一堆代码。没有代码级根因与反证实验两样的 Plan 不批。

4 Skill: 工具链与各自的真实战绩

工具	是什么	本项目战绩
advisor 二次评审	实现前 / 收尾前请独立模型审方案	R9: 否决「实时气泡里也放正文」的初稿——正文已被投影器写成正式消息,气泡再放必然重复渲染;改为只放 thinking + 工具指示 (见 §5.2)
memory 持久化	跨会话沉淀非显然事实	沉淀了「claudecode 单轮根因与 stream-json 多轮语义」「房间历史不加载的根因」「编排架构与分工边界」三份,后续会话零成本续接
/verify 浏览器实测	Playwright 驱动真实 UI 验收	R8(文件 / 终端)、R9(实时气泡 22/22 采样命中)、demo 预演;还反向发现了「旧房间历史造成检测假阳性」的演示风险
pre-commit 门禁	husky:golangci-lint + go test ./... + eslint	每个 R commit 都过门禁;R8 曾被 gosec/errorlint 拦下返工 (路径穿越标注、errors.As 改写)
事件流 dump 调试	直接查编排事件表做定量诊断	R9 立项依据:dump 213 个 agent_output 事件,发现 13 个有效信号跨 27 秒增量到达——证明「数据是流式的,只是 UI 没消费」,方案因此定为纯前端

4.1 内置协作机制的使用 (Claude Code / memoh)

上述工具之外,协作流程还系统性使用了两层「内置机制」——与开发助手 (Claude Code) 协作的机制,以及与平台内 Agent(memoh bot) 协作的机制。后者与产品功能同源: 我们既在开发这些协作机制,也在用它们与 Agent 协作。

层	机制	本项目用法
Claude Code	Plan Mode(计划审批)	复杂改动先出 Plan、人批准后执行;~/ <code>.claude/plans/</code> 留存 8 份真实 plan 文件
Claude Code	Subagent(子代理)	用低成本模型分担机械工作: 本文档的产品 / 技术精简节与会话时间线即由 3 个 haiku 子代理压缩产出
Claude Code	Memory(持久记忆)	跨会话沉淀根因与架构决策 (MEMORY.md 索引 + 三份记忆文件), 新会话零成本续接
Claude Code	Skills(/verify、/code-review)	浏览器实测与代码评审作为固定收尾动作
Claude Code	CLAUDE.md Rules	全局硬约束每次会话强制加载 (§2.1)
Claude Code	AskUserQuestion 检查点	关键分叉 (封面方案、引用处理、交付物结构) 以选项形式交人决策
memoh	pin 置顶长期上下文	开发与测试期间用置顶约束 bot 行为, 关键约定跨轮稳定生效
memoh	显式 @ 调度	测试编排时以 @ 点名压制 LLM 重新分工, 验证「用户说了算」
memoh	DeferredToolApproval	高危工具人工审批, 与开发侧「确认后执行」同构
memoh	共享工作目录	验收方式: 当场在群工作区终端跑 <code>cargo test</code> 验证 Agent 交付物

5 三个深度协作用例

5.1 案例一 (R7): 同一个 bot 单聊能跑、编排零产出——三层根因的逐层剥离

现象: 群聊编排里 cc 任务「成功」但只有 system 事件, 无任何文本产出。三个回合剥了三层:

- 凭证层 (R4): 编排路径只读环境变量, 容器里没有 ANTHROPIC_API_KEY -> 把单聊的 DB 凭证解析 (ResolveCredentialsForFramework) 注入编排 Provider, 两条路同源;
- 工作目录层 (R4.1): 编排曾解析出 bot 嵌套容器内的路径, 而 CLI 子进程跑在 server 侧根本看不到 -> 改为 `os.TempDir()/memoh_agenthub_rooms/<roomID>` 的每房间共享 host 目录——顺手把用户「挂载目录贡献给全群」的产品想法一并落地;
- 模型层 (R7, 最隐蔽): 凭证对了、目录对了, 还是零产出。日志比对发现编排路径没传 `--model`, CLI 默认 Claude 系模型, DeepSeek 端点不识别 -> 静默地只回 system 事件。修法: 执行时读 bot 的 `provider_ext["claudecode"]` 把它单聊用的模型带给 CLI(`claudecode.go:54` 注释完整记录了这条因果链)。

方法论要点: 每层修完都实测复现, 不见产出不收工——否则修完第一层就会错误地宣布胜利两次。

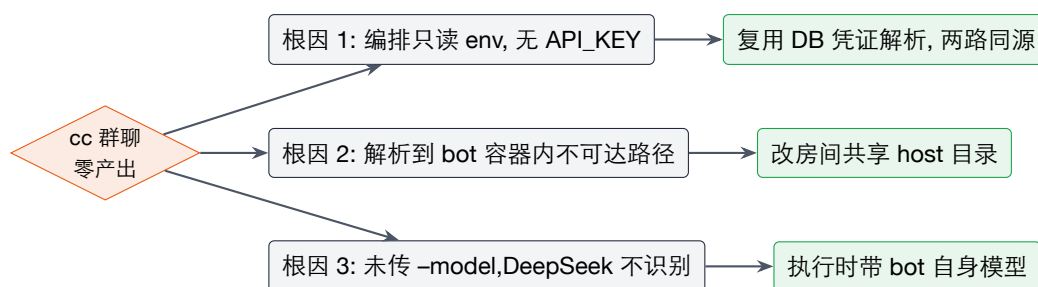


Figure 2: 图 2: 案例一三层根因与对应修复 (逐层实测, 不见产出不收工)。

5.2 案例二 (R9):AI 评审纠正 AI 的方案

R9 目标是群聊编排实时可见。初稿方案把「累积正文」也放进实时气泡。advisor 评审指出: 投影器已把 text 事件写成正式房间消息, 气泡再放正文 = 同一内容双份渲染, 且任务结束时草稿与正式消息的「交接闪烁」无法干净处理。最终设计: 气泡只承载被投影器故意丢弃的过程信号 (**thinking** / 工具调用), 正文永远只走投影——气泡与消息职责互斥, 交接天然干净。实现零后端改动, 守住「勿增实体」。这种「AI 审 AI」机制成本极低, 却能抓到结构性问题。

5.3 案例三: 一次「像 bug 的非 bug」——证据链排除法

交付前发现仓库根目录多出 `lru-cache/` (完整 Rust 工程)。第一反应像是「编排 Agent 把仓库当工作目录写了」——若属实是严重缺陷。AI 没有直接改代码, 而是做了完整证据链:

1. 编排日志: `work_dir=/tmp/memoh_agenthub_rooms/<roomID>`, 该容器目录里确实有全部 Agent 产物;
2. 单聊路径: `/tmp/memoh_cli_bots/<botID>`, 同样不在仓库;
3. server 进程 `cwd = /`, 排除 `os.Getwd()` 兜底落仓库的可能;
4. bot 嵌套容器挂载表: 无仓库挂载, 写不进来;
5. 文件指纹: 连字符命名的 cargo new 完整工程 (容器内产物是下划线散文件), 时间戳早于当日所有编排 run。

结论: 宿主机上某个以仓库为 `cwd` 的进程所留, 与平台无关。处置: 不改任何功能代码, 加一行 **gitignore**。这是「AI 不瞎改」的范本: 像 bug ≠ 是 bug, 动状态前先让证据收敛。

6 失败与教训 (同样是沉淀)

1. 未提交代码被 **git restore** 清掉: R1-R3 部分改动尚未提交时被协作者误 restore, 半天工作重做。教训固化为规则「完成一个 commit 一次」, 此后 R4-R9 每回合独立成 commit, 再未发生损失。
2. AI 滥用后台轮询 **shell** 被用户纠正: 早期 AI 倾向起多个长驻轮询 shell 盯构建, 占资源且无反馈。用户两次点名后改为「单次快命令 + 必要时单个后台任务」, 写入工作习惯。
3. **macOS Docker** 开发环境的运维坑 (沉淀进 memory 与 runbook): 卷挂载 `fsnotify` 不可靠 -> 容器内 touch 触发重建; `kill go build` 会误杀 air 自身构建造成僵尸链接器堆积 -> 只清理僵尸、保最新; Go 链接慢 -> 等单次构建完成, 不并发触发。

7 量化记录 (可在 git log 验证)

回合	Commit	内容	规模
R1-R3	f2bd672	memoh 编排 Provider、按框架分派、pin 长期上下文	6 文件 +380/-7
R4	b3ae212	编排 claudecode 复用通用设置 DeepSeek 凭证	4 文件 +58/-18
R4.1	144608d	每房间共享 host 工作目录	3 文件 +39/-3
R5	850999c	单聊回复逐字流式渲染	1 文件 +58/-4
R6	ad27637	激活 LLM Planner 动态分派 (显式 @ 仍走规则)	2 文件 +84/-18
R7	247adc0	编排 cc 携带 bot 自身模型 (修零产出)	2 文件 +69/-39
R8	0da297a	群聊工作区: 文件浏览 + 浏览器终端	3 文件 +453
R9	ea6e394	群聊编排运行中实时过程气泡	1 文件 +146

Table 2: 合计 8 个功能 commit, 约 1.9k 行净增, 每个均通过 lint+test 门禁、均有浏览器实测记录,commit message 完整记载根因与验证方式 (可直接 git show 复核)。

8 真实协作会话档案

本节统计直接来自本机 ~/.claude/projects 下的 Claude Code 会话日志 (jsonl), 用解析脚本聚合得到, 可复核。比赛周期为 2026-05-20 至 06-10, 本节取本机留存的 06-05 至 06-10 会话作为代表性样本。

总量:19 个有效会话 · 277 个用户回合 · 5459 个助手回合 · 2582 次工具调用 · 跨 2026-06-05 至 06-10 共 5 个活跃天 · 3 条分支 (feature/claude-remote8163 早期 / merge/main-incremental 合并 / fix/claude-code-integration 后期)。
平均每个用户回合触发约 20 个助手回合与 9 次工具调用——这正是「人定目标、AI 多步执行」分工的量化体现。

8.1 协作时间线

- 第一阶段 (06-05 – 06-07 中,feature/claude-remote8163): 聚焦 Claude Code SDK 集成, 解决单轮记忆缺失——用 stream-json 拼接多轮上下文、设最大对话轮次, 改造假思考动画为可点击展开的真实思考 / 工具调用链, 调试流式超时与环境变量透传。对应 R1-R5。
- 第二阶段 (06-07 中,merge/main-incremental): 与 main 增量合并, 逐块处理框架选择器 (claude / codex / memoh 三者兼容)、Provider 与凭证配置冲突, 每步完成后测试验证。
- 第三阶段 (06-08 – 06-09,fix/claude-code-integration 早期): 集中实现 orchestrator 多 Agent 编排、凭证隔离与模型配置、工作区文件浏览、富媒体渲染。对应 R4 凭证 / 模型、R8 工作区。
- 第四阶段 (06-09 – 06-10, 后期): 完成 memoh / Claude Code / Codex 三 Provider 验证, 解决实时过程气泡、工作区共享、群聊默认 Agent、国际化等生产级问题, 每个修复前都实机验证。对应 R7 / R8 / R9 完整端到端。

8.2 会话强度 (工具调用数, 按时间)

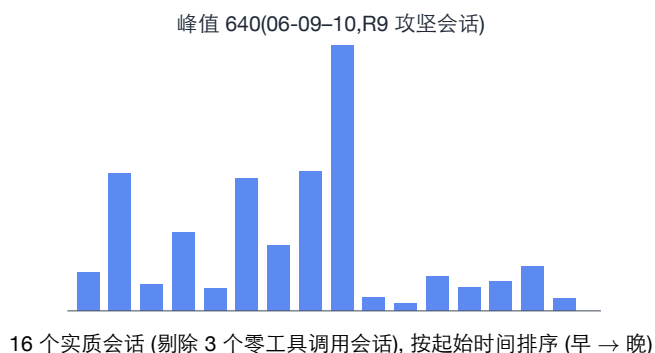


Figure 3: 每个实质会话的工具调用数随时间分布: 后期编排集成 (R7–R9) 的会话明显更密集。

8.3 代表性人类指令 (原文摘录)

以下原话取自会话日志, 体现「开发者作为导航员」的协作风格 (规划先行、增量小步、强约束、打断纠偏):

- 先不要做和我聊聊
- 你先研究一下怎么做先不要做和我聊聊
- 我们一个一个来吧你先 *grep* 一下「Agent 正在思考中...」
- 你的计划还可以但是我们分几步每次完成后我可能都想要看看是不是 *work*
- 我们先多看看 *body* 以及 *memoh* 智能体上下文是怎么做的
- 不改 *chat* 只修 *agenthub*
- 应该使用 *ANTHROPIC_AUTH_TOKEN* 这个 */env* 对容器内不起作用
- 如果你知道而且和目前的想法一致就按照你的如果不清楚你研究一下
- 我计划要和 *main merge* 一下但是有很多地方有冲突希望你研究一下给个方案
- 看看 *memoh* 怎么做的 *codex* 怎么做的 *claude* 怎么做的主要是 *merge* 加注释让三个都可以跑
- 修复吧修复前写个 *plan*
- 实时改动的还是需要我重启服务你不要动

8.4 协作模式观察

- 规划先行: 大改动前先要求研究现状、对比方案 (「先不要做和我聊聊」「研究一下」), 而非直接上手;
- 增量小步、每步验证: 「一个一个来」「每次完成后看看是不是 *work*」, 拒绝一次性大改;
- 事实驱动调试: 用日志、git 历史、容器实际状态驱动决策, 假设与事实不符即纠正;
- 硬约束边界清晰: 「不改 *chat* 只修 *agenthub*」「你不要动 (我手动重启)」防跨域改动;
- 让 AI 先调研再实现: 不确定时不直接给答案, 而是「你研究一下」「看看……怎么做的」;
- 严格的 **review/测试/commit** 闭环: 改前写 *plan*、改后 *code review*、验证通过才 *commit*。

9 产品设计概述 (精简)

本节为产品设计要点的精简版，完整论证见《产品设计文档》(agenthub-product-design.tex)。

9.1 背景与群聊心智

AgentHub 采纳 IM(群聊) 范式做多 Agent 协作, 以克服两类现有工具的结构性缺陷: 单助手对话式 (如 ChatGPT) 多任务并行时上下文互相污染、无法表达分工; workflow 编辑器式 (如 Coze / Dify) 门槛高, 违背「用说话的方式干活」的直觉。设计判断是复用群聊心智: 把 AI Agent 映射成「联系人 / 群成员」, 单聊用于 1v1 指派、群聊用于多 Agent 协作、群文件即共享工作目录、群置顶即长期上下文约束——协作模型无需再教学。主场是 **coding**: 代码有清晰接口契约, 契约驱动的协作范式优势最突出。

9.2 多 Agent 失效模式与应对

失效模式	AgentHub 的应对	状态
对话漂移	置顶消息作为长期上下文注入每轮; 近 20 条滚动历史进 Planner 与子任务 prompt(pinnedRoomContext)	已实现
幻觉级联	Human-in-the-loop:Plan 等关键阶段人工及时纠偏;AI 自动审查规划中	已实现
角色漂移	固定 skills 与 MCP tool + 工具白 / 黑名单 + System Prompt 防越权	已实现
安全漂移	单聊高危工具审批拦截 (DeferredToolApproval); 群聊策略推进中	部分实现
操作幻觉 / 活锁	CLI --max-turns 15 + 超时 + 重试预算; 单 bot TextLoopGuard/ToolLoopGuard	已实现
上下文腐化	单聊 pruneOldToolResults + internal/compaction 压缩; 预评估路由规划中	已实现
Token 成本爆炸	默认单 Agent、按需开多 Agent(单 @ 不升级编排); 每任务精确预算规划中	已实现

9.3 目标用户与典型场景

目标用户: 需借助多个 AI Agent 完成开发 / 产出任务的工程师与技术产品人, 想指挥和监督工作而非每行代码亲自写。场景:**A** 单点任务 (单聊流式生成, 可演示)、**B** 复合任务 (群聊自动编排拆 DAG 并行分派, 可演示)、**C** 用户主导流水线 (显式 @ 按序生成依赖链, 可演示)、**D** 多轮迭代 (置顶约束跨轮生效, 可演示)、**E** 契约驱动全栈协作 (扁平 DAG + 共享目录, 核心场景已通)。

9.4 核心体验决策

- 过程必须可见: 单聊 WebSocket 流式草稿气泡; 群聊编排每步落为有序事件,1.5s 级临时气泡, 任务结束让位正式产出;
- 置顶即长期上下文: 复用 IM 置顶语义作长期记忆, 标注「务必优先考虑」注入每轮, 缓解约束漂移;

3. 用户点名优先: 显式 @ 走确定性规则, LLM 只在「只给目标不给分工」时介入;
4. 共享工作目录: 同房间 CLI Agent 在同一目录执行, 产物天然互见, 可当场 cargo test 验证;
5. 富媒体卡片: 代码块 / Diff 卡 / 工具调用卡 / 思考折叠 / Artifact 预览 / 附件块;
6. 失败被设计过: 降级消息不连坐、指路型中文错误、静默重试成功不打扰。

10 系统架构与技术实现 (精简)

本节为技术实现要点的精简版, 完整 DDL / 时序 / API 见《技术文档》(agenthub-technical-design.tex)。

10.1 分层架构

六层, 依赖方向严格单向: handlers → agenthub → orchestrator ← providers。领域包 orchestrator 不 import 业务包, 可独立单测、可替换宿主。



Figure 4: 图 3: 分层组件图 (与《技术文档》同源)。

10.2 数据模型与事件溯源

编排域五表: runs(编排生命周期)、tasks(DAG 节点)、task_deps(显式依赖边)、task_attempts(每次执行, 与 tasks 分离以无损支持重试)、run_events(append-only 事件流, seq 在 run 内单调, 是投影 / 实时 / 审计 / 回放的唯一事实源)。幂等键 sha256(runID|taskID|attemptNo|description) 防重复派发。

10.3 状态机与调度

Run: draft -> planning -> dispatching $\rightleftharpoons$ collecting -> completed/failed/cancelled。Task: pending -> ready -> running -> succeeded/failed, 失败按预算回炉 ready, 上游失败则 blocked。

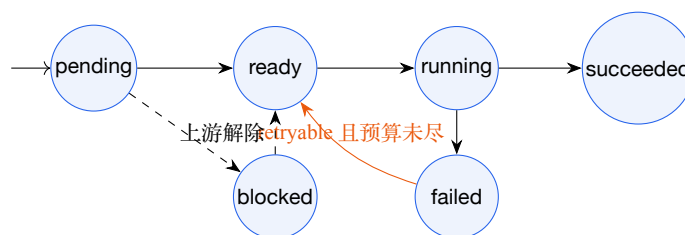


Figure 5: 图 4:Task 生命周期状态机。失败降级 = 只阻塞下游、不连坐旁路分支。

调度由幂等入口 ReconcileRun 驱动, 无常驻调度线程; 任意入口 (StartRun / 异步完成回调 / 前端轮询) 都能把 DAG 推到底。生产配置 MaxParallelPerRun=3、MaxParallelPerAgent=1 (CLI Agent 共享目录, 串行写)。崩溃自愈: failTimedOutTasks 比较 attempt 开始时间与进程启动时间, 重启后第一次 reconcile 即把中断的 running attempt 判 interrupted 回炉。

10.4 双层 Planner 与适配器

决策序: 显式 @ -> directMentionPlan(确定性); 否则 LLM Planner 拆 DAG, 失败回落默认五段 DAG。LLM 输出防御性解析 (抽 JSON、改派幻觉 agent、Kahn 检测依赖环、成环整体拒绝), validatePlan 兜底。适配器统一三方法 Name/Capabilities/Execute; CLI 型用 CLIRunner 跑子进程 + 跨 chunk 行缓冲 NDJSON 解析; Memoh 用接口反转避免包环。凭证解析链 bot provider_ext -> 通用设置 anthropic 兼容 Provider -> 环境变量, 与单聊同源, 带 --model 是 DeepSeek 端点能跑的关键。

10.5 设计 vs 实现对照

设计机制	代码现实	状态
任务拆解 + 并行调度 + 聚合	LLM / 规则双层 Planner + DAG 引擎 + 投影	已实现
失败降级、重试预算、超时	completeTaskAttempt / failTimedOutTasks	已实现
崩溃自愈	进程启动时间比较	已实现
多轮上下文 / 置顶长期记忆	room_history(置顶 + 滚动) 注入	已实现
跨模型族	ClaudeCode + Memoh(DeepSeek);Codex 就绪未启用	部分实现
显式 @ > LLM 决策	hasExplicitAgentMention -> directMentionPlan	已实现
事件溯源 / 审计 / 回放	agent_hub_run_events(seq 单调)	已实现
Plan 优先交互(确认闸)	run 自动派发,「规划」为执行后叙事	路线图
接口契约(生成 / 传递 / 验证)	无; 共享目录 + DAG 依赖近似	路线图
独立 Verification Agent	verify 仅普通子任务	路线图
多级 Sub-Orchestrator + 预评估路由	仅扁平 DAG;parent_task_id 预留未用	路线图
Markov 安全监控 / 活锁检测	仅 CLI --max-turns + 超时	路线图
每任务硬性 token 预算	无	路线图

10.6 已知局限

投影幕等的进程内边界(可下沉到 DB 唯一约束)、轮询实时性下限 1.5s(可迁 WS)、代码冲突靠串行规避无自动合并、Codex 适配器就绪未启用、工作区终端为房主信任模型、契约驱动 / 多级编排 / 安全监控为路线图。

11 沉淀给后来者的协作清单

1. 把硬约束写进会话级 Rules, 别依赖每次口头重复; 约束要短、可判定(如「勿增实体」)。
2. 复杂改动先要 Plan, Plan 必须含代码级根因与反证实验——没有这两样的 Plan 不批。
3. 一回合一个目标一个 commit; commit message 写「为什么」, 它就是开发日志。
4. 验收以「真实运行的观测」为准(浏览器 / 日志 / 事件流), 不接受「代码看起来对」。
5. 重要方案让第二个模型评审一遍, AI 审 AI 的成本极低、抓到的都是结构性问题。
6. 像 bug 的现象先建证据链再动手; 排除法的结论(「不改」)和修复同样有价值。
7. 跨会话的事实(环境坑、架构决策、根因)写入持久 memory, 不靠人脑转述。

12 结语

通过 Rules、Spec/Plan、Skill 工具链与回合制流程的组合, AgentHub 项目实现了人与 AI 的高效协作: 开发者提供策略与审核, AI 执行调研与实现, 两者在回合制节奏中相互校正, 既保证代码质量

也保证迭代速度。开发者在与 CodeGen 模型协作时应担任导航员, 负责规划与 guardrail⁴; 本项目 19 个会话、277 个用户回合、2582 次工具调用的真实记录进一步印证了这一点。未来的 AI 开发仍需遵循「计划先行、约束先行、验证先行」的原则, 才能让 AI 真正成为可靠的开发伙伴。

参考文献

- [1] Stack Overflow Blog. 关于 AI 结对编程的实践讨论: 开发者应担任「导航员」, 负责规划与审核, AI 负责生成代码 (「AI 的工作是快, 你的工作是好」)。
- [2] Root Cause Analysis 方法论: 系统性流程——定义问题、收集数据、分析因果、提出并验证解决方案 (工程质量管理通用实践)。
- [3] Guardrails for human-AI pair programming: 开发者通过明确边界与验收测试约束生成式模型行为, 避免幻觉与越权调用。

⁴见参考文献 [1][3]。